

Semana 2: Hablando en ceros y unos — Binario, codificación y representación

Guía completa de la sesión

Visión general

Duración total	3 horas (2 h cátedra + 1 h laboratorio analógico)
Objetivo central	Que los estudiantes comprendan que toda información — números, texto, imágenes, sonido — se reduce a secuencias de 0 y 1, y que puedan convertir entre sistemas numéricos
Idea ancla	Un computador solo distingue dos estados: encendido/apagado. Todo lo demás es <i>convención</i> — un acuerdo humano sobre qué significa cada patrón de bits
Prerrequisito	Semana 1 (concepto de computador, componentes, instrucciones)
Materiales	Proyector, pizarra, cuentas/mostacillas de dos colores (o lápices), hilo/limpiapipas, tablas ASCII impresas, copias del Control 1
Conexión con Semana 1	La semana pasada vimos que la Memoria almacena “valores” y la ALU opera con “números”. Hoy respondemos: ¿cómo se almacenan esos valores <i>físicamente</i> ?

PARTE 1: CÁTEDRA (120 minutos)

Bloque A — Apertura: el truco de la carta binaria (15 min)

Actividad demostrativa (hacer frente al curso):

Preparar 5 tarjetas grandes (hojas tamaño carta) con puntos en una cara:

Tarjeta 1: 16 puntos

Tarjeta 2: 8 puntos

Tarjeta 3: 4 puntos

Tarjeta 4: 2 puntos

Tarjeta 5: 1 punto

Pedir a un voluntario que piense un número del 0 al 31. El docente, dándole la espalda, pide al curso que volteen las tarjetas cara arriba o cara abajo para representar ese número. Luego, el docente se da vuelta, “lee” las tarjetas y adivina el número.

Revelar: las tarjetas son potencias de 2. Cada tarjeta cara arriba = 1, cara abajo = 0. Es un número binario de 5 bits.

Ejemplo: Si el número es 19 $\rightarrow$ tarjetas: $16+2+1 =$ cara arriba, $8+4 =$ cara abajo $\rightarrow 10011$

Transición: “*Esto que acaban de hacer es exactamente lo que hace la memoria de un computador. Cada celda de memoria es como una tarjeta: encendida o apagada. Y con eso, puede representar cualquier número.*”

Bloque B — Sistemas numéricos (35 min)

El sistema decimal (lo que ya saben) — 5 min

- Base 10: diez símbolos (0–9)
- Cada posición vale una **potencia de 10**
- Ejemplo: $347 = 3 \times 10^2 + 4 \times 10^1 + 7 \times 10^0 = 300 + 40 + 7$

Punto clave: el sistema decimal no tiene nada de especial — solo lo usamos porque tenemos 10 dedos. Si tuviéramos 8 dedos, usaríamos base 8.

El sistema binario — 15 min

- Base 2: dos símbolos (0 y 1)
- Cada posición vale una **potencia de 2**
- La unidad mínima de información: el **bit** (binary digit)
- 8 bits = 1 **byte**

Tabla de referencia (proyectar y dejar visible):

Posición	7	6	5	4	3	2	1	0
Valor	128	64	32	16	8	4	2	1

Conversión binario $\rightarrow$ decimal:

$$10110 = 1 \times 16 + 0 \times 8 + 1 \times 4 + 1 \times 2 + 0 \times 1 = 16 + 4 + 2 = 22$$

Conversión decimal $\rightarrow$ binario (método de la resta):

Ejemplo: convertir 45 a binario. - ¿Cabe 128? No. ¿64? No. ¿32? Sí $\rightarrow 45 - 32 = 13$, anoto 1 - ¿16? No $\rightarrow$ anoto 0 - ¿8? Sí $\rightarrow 13 - 8 = 5$, anoto 1 - ¿4? Sí $\rightarrow 5 - 4 = 1$, anoto 1 - ¿2? No $\rightarrow$ anoto 0 - ¿1? Sí $\rightarrow 1 - 1 = 0$, anoto 1 - Resultado: 101101

Ejercicio rápido en la pizarra (3 min, participación del curso): - Convertir 11001 a decimal $\rightarrow$ 25 - Convertir 13 a binario $\rightarrow$ 1101

¿Por qué binario? — 5 min Un computador no “elige” usar binario por gusto. Lo hace porque es **físicamente simple**: un transistor tiene dos estados (conduce / no conduce), un condensador tiene carga (1) o no la tiene (0), un cable tiene voltaje alto o bajo.

Con solo dos estados se minimiza el error: si hay ruido eléctrico, es mucho más fácil distinguir “alto vs. bajo” que distinguir entre 10 niveles diferentes.

Analogía ecológica: Es como clasificar el hábitat en “presente/ausente” vs. una escala de 1 a 10 de calidad. La primera es más tosca pero mucho más robusta ante el error de observación.

El sistema hexadecimal (breve) — 10 min

- Base 16: dígitos 0–9 y letras A–F (A=10, B=11, ..., F=15)
- Cada dígito hexadecimal = exactamente 4 bits
- Se usa como **notación compacta** para binario

Hex	Binario	Decimal
0	0000	0
5	0101	5
A	1010	10
F	1111	15
FF	11111111	255

Ejemplo práctico: El color blanco en pantalla es #FFFFFF $\rightarrow$ tres bytes: FF FF FF $\rightarrow$ tres veces 255 $\rightarrow$ rojo 255, verde 255, azul 255. (*Anticipo de la sección de imágenes.*)

¿Dónde van a ver hexadecimal? Colores en CSS/HTML, direcciones de memoria, direcciones MAC de red.

Bloque C — Representando texto: ASCII y UTF-8 (20 min)

El problema — 3 min Un computador solo almacena números (secuencias de bits). Para almacenar texto, necesitamos un **acuerdo**: ¿qué número corresponde a qué letra?

Ese acuerdo se llama **codificación de caracteres**.

ASCII — 10 min

- American Standard Code for Information Interchange (1963)
- 7 bits → 128 caracteres posibles (0–127)
- Incluye: letras mayúsculas (A=65, B=66, ..., Z=90), minúsculas (a=97, ..., z=122), dígitos (0=48, ..., 9=57), signos de puntuación y caracteres de control

Tabla parcial (proyectar):

Carácter	Decimal	Binario
A	65	01000001
B	66	01000010
Z	90	01011010
a	97	01100001
0 (dígito)	48	00110000
espacio	32	00100000

Observación clave: entre ‘A’ y ‘a’ hay exactamente 32 de diferencia. Eso no es casualidad — basta cambiar un solo bit (el bit 5) para pasar de mayúscula a minúscula. Diseño inteligente.

Pregunta al curso: “¿Qué pasa con la ñ? ¿Y con los acentos?” → No están en ASCII estándar. ASCII fue diseñado para el inglés. Para el español necesitamos algo más grande.

UTF-8 — 7 min

- Unicode: catálogo universal de ~150.000 caracteres (todos los idiomas, emojis, símbolos)
- UTF-8: la codificación más usada en internet
- Usa 1 a 4 bytes por carácter
- Los primeros 128 caracteres de UTF-8 son idénticos a ASCII (compatibilidad)
- La ñ es el código 241, la á es 225

Ejemplo del mundo real: cuando ven “Ã±” en vez de “ñ” en una página web, es un error de codificación — el navegador interpretó los bytes con la tabla equivocada.

Dato: el emoji (árbol) es el carácter Unicode U+1F333, codificado en 4 bytes en UTF-8. Sí, un emoji pesa más que una letra.

Bloque D — Representando imágenes: píxeles y RGB (20 min)

Píxeles — 7 min

- Una imagen digital es una **grilla de puntos** (píxeles)
- Cada píxel tiene un color
- Resolución = ancho × alto en píxeles (ej: 1920×1080 = ~2 millones de píxeles)

Demostración: proyectar una foto de paisaje valdiviano. Hacer zoom progresivo hasta que se vean los píxeles individuales. *“Toda foto, por hermosa que sea, es en el fondo una tabla de números.”*

Color RGB — 8 min

- Cada píxel se describe con tres valores: **Rojo, Verde, Azul** (RGB)
- Cada canal: un byte (0–255)
- 3 bytes por píxel = 24 bits
- Total de colores posibles: $256 \times 256 \times 256 = \mathbf{16.777.216}$

Ejemplos:

Color	R	G	B	Hex
Negro	0	0	0	#000000
Blanco	255	255	255	#FFFFFF
Rojo puro	255	0	0	#FF0000
Verde bosque	27	94	32	#1B5E20
Azul océano	21	101	192	#1565C0

Cálculo rápido: ¿Cuánto pesa una foto de 12 megapíxeles sin comprimir? $12.000.000 \text{ píxeles} \times 3 \text{ bytes} = 36.000.000 \text{ bytes} = \mathbf{36 \text{ MB}}$. Por eso existen JPEG y PNG: compresión.

Conexión con ecología — 5 min

- Las imágenes satelitales (Landsat, Sentinel) son exactamente esto: grillas de píxeles.
- Pero en vez de RGB, tienen **más canales** (infrarrojo cercano, infrarrojo de onda corta, etc.)
- Clasificar uso de suelo desde un satélite es, en el fondo, clasificar patrones de números en una tabla gigante — algo que un computador hace muy bien.

Bloque E — Representando sonido: muestreo y cuantización (10 min)

La idea — 5 min

- El sonido es una **onda continua** (variación de presión del aire)
- Para digitalizarlo: medir la amplitud de la onda a intervalos regulares (**muestreo**)
- Cada medición se convierte en un número (**cuantización**)

Parámetros clave: - **Frecuencia de muestreo:** cuántas veces por segundo se mide (CD = 44.100 Hz $\rightarrow$ 44.100 muestras por segundo) - **Profundidad de bits:** cuántos bits por muestra (CD = 16 bits $\rightarrow$ 65.536 niveles posibles)

Conexión con ecología — 5 min

- El monitoreo acústico de biodiversidad (AudioMoth, grabadoras pasivas) hace exactamente esto: muestrea el sonido ambiente y lo almacena como secuencias de números.
- Los algoritmos de identificación de cantos de aves trabajan sobre estas secuencias numéricas.
- Mayor frecuencia de muestreo = más detalle = archivos más grandes. Un AudioMoth a 48 kHz / 16 bits genera ~5 MB por minuto de grabación.

Bloque F — El bit como unidad fundamental + cierre (20 min)

Unidades de información — 10 min

Unidad	Equivalencia	Ejemplo concreto
1 bit	0 o 1	Una respuesta sí/no
1 byte	8 bits	Un carácter ASCII
1 kilobyte (KB)	~1.000 bytes	Un párrafo de texto
1 megabyte (MB)	~1.000 KB	Una foto JPEG
1 gigabyte (GB)	~1.000 MB	~250 canciones MP3
1 terabyte (TB)	~1.000 GB	~500 horas de video

Nota técnica (para los curiosos): 1 KB = 1.024 bytes exactamente (2^1), pero en la práctica se usa 1.000 como aproximación. Los fabricantes de discos duros usan 1.000 (les conviene); los sistemas operativos usan 1.024 (es correcto técnicamente). Por eso un disco de “500 GB” muestra ~465 GB en el computador.

Recapitulación: todo es bits — 5 min

Tipo de dato	Cómo se codifica	Acuerdo/estándar
Números	Sistema binario (posicional)	Complemento a 2, IEEE 754
Texto	Cada carácter → un número → binario	ASCII, UTF-8
Imágenes	Cada píxel → 3 números (RGB) → binario	JPEG, PNG
Sonido	Muestras periódicas → números → binario	WAV, MP3

Mensaje central: el hardware solo distingue dos estados. **Todo lo demás es convención.** Los estándares (ASCII, RGB, WAV) son acuerdos humanos sobre cómo interpretar las secuencias de bits. Sin el acuerdo, los bits no significan nada.

Puente al laboratorio — 5 min *“Ahora van a fabricar un mensaje usando el código más antiguo de la informática: van a escribir sus iniciales en binario, materializadas en una pulsera de cuentas de dos colores. Y después van a decodificar las de sus compañeros — sin que les digan qué dice.”*

PARTE 2: LABORATORIO ANALÓGICO (60 minutos)

“Pulseras Binarias”

Preparación previa (docente)

Materiales (para 30 alumnos)

Material	Cantidad	Notas
Cuentas/mostacillas de color A (ej: oscuras = 1)	~300	Pueden ser cuentas, porotos pintados, o cuadrados de cartulina
Cuentas/mostacillas de color B (ej: claras = 0)	~300	
Hilo, limpiapipas, o cordel	30 tramos (~25 cm cada uno)	Limpiapipas es más fácil de manipular en bancas fijas
Tabla ASCII impresa	15 copias (1 cada 2 estudiantes)	Solo caracteres imprimibles: 32–127
Tabla de conversión binaria (poderes de 2)	15 copias	Proyectar también

Alternativa sin cuentas (más simple para auditorio): - Usar **papel cuadriculado** y lápices de dos colores. - Cada cuadro = 1 bit. Colorear = 1, dejar en blanco = 0. - Funciona igual pedagógicamente, sin la artesanía manual.

Recomendación para bancas fijas: la alternativa de papel cuadriculado es más práctica. Las cuentas tienden a caerse y rodar. Si se usan cuentas, los limpiapipas son mucho mejor que hilo (se mantienen rígidos sobre la banca).

Convención de colores (proyectar)

cuenta oscura = 1 (encendido, hay carga, voltaje alto)

cuenta clara = 0 (apagado, sin carga, voltaje bajo)

Cada letra se codifica en **8 bits (1 byte)**, leyendo de izquierda a derecha, del bit más significativo al menos significativo.

Desarrollo de la actividad

Fase 1 — Codificación individual (20 min) **Tarea:** Codifica tus dos iniciales (nombre + apellido) en binario usando la tabla ASCII.

Paso a paso:

1. Buscar cada inicial en la tabla ASCII → obtener el número decimal. Ejemplo: “H” = 72, “P” = 80
2. Convertir cada decimal a binario de 8 bits.
 - 72 = 01001000
 - 80 = 01010000
3. Ensartar las cuentas en el limpiapipas (o colorear en el papel cuadriculado):
 - Empezar por la primera letra, bit 7 (el de la izquierda)
 - Poner un separador (cuenta de otro color, o espacio en blanco) entre las dos letras

Resultado: una pulsera de 16 cuentas + 1 separador = 17 cuentas.

Mientras trabajan: el docente circula y verifica que la conversión decimal → binario esté bien. Errores comunes: olvidar los ceros a la izquierda (72 no es 1001000 en 7 bits — son 8 bits: 01001000).

Fase 2 — Intercambio y decodificación (15 min) **Tarea:** Intercambia tu pulsera (o papel) con un compañero de otro grupo. Decodifica sus iniciales.

Paso a paso:

1. Leer las cuentas (o cuadros) de izquierda a derecha → escribir la secuencia de 0s y 1s.
2. Separar en dos bytes de 8 bits.
3. Convertir cada byte de binario → decimal.
4. Buscar en la tabla ASCII → obtener la letra.

Regla: NO se puede preguntar al compañero cuáles son sus iniciales. La decodificación es la prueba de que el código funciona.

Verificación: comparar el resultado con las iniciales reales del compañero. Si no coinciden → buscar el error juntos (¿fue en la codificación o en la decodificación?).

Fase 3 — Extensión: mensaje secreto entre grupos (15 min) **Tarea (por grupos de 5):** Codificar una **palabra completa** (máximo 5 letras, sin tildes ni ñ) en binario y enviarla a otro grupo para que la decodifique.

Procedimiento: 1. El grupo elige una palabra (ej: “BOSQUE” → no, tiene 6 letras → “FLORA”, “CELDA”, “ARBOL”). 2. Cada integrante codifica una letra. 3. Ordenan las letras y entregan la secuencia completa al grupo receptor. 4. El grupo receptor decodifica y anuncia la palabra.

Palabras sugeridas (con contexto ecológico, todas de 5 letras, sin caracteres especiales):

Grupo emisor $\rightarrow$ receptor	Palabra
Grupo 1 $\rightarrow$ 2	FLORA
Grupo 2 $\rightarrow$ 3	CELDA
Grupo 3 $\rightarrow$ 4	NIDOS
Grupo 4 $\rightarrow$ 5	COSTA
Grupo 5 $\rightarrow$ 6	LENGA
Grupo 6 $\rightarrow$ 1	HUMUS

(Entregar en sobre cerrado a cada grupo — solo la palabra que deben codificar.)

Fase 4 — Discusión breve (10 min) Preguntas guía:

1. “¿Cuántas cuentas necesitaron para una sola letra?” $\rightarrow$ 8. Y un emoji necesita 32 (4 bytes). La información tiene un *costo* en espacio.
2. “¿Qué hubiera pasado si cada grupo usara una convención de colores diferente?” $\rightarrow$ Caos. Sin un estándar compartido, la comunicación es imposible. Por eso existen ASCII y UTF-8.
3. “¿Y si quisiéramos codificar la ñ?” $\rightarrow$ No está en ASCII básico. Necesitamos Unicode/UTF-8 $\rightarrow$ más bits por carácter $\rightarrow$ más cuentas.
4. **Conexión con la Semana 1:** “La semana pasada, la Memoria guardaba ‘valores’ en celdas. Ahora saben cómo se ven esos valores físicamente: secuencias de bits. Y la ALU, cuando sumaba, operaba sobre esas secuencias.”
5. **Adelanto Semana 3:** “Si cada letra cuesta 8 bits, un libro de 500 páginas cuesta millones de bits. ¿Se puede ser más eficiente? Eso es compresión — y tiene que ver con la información y la sorpresa. La próxima semana vamos a medir la sorpresa con una fórmula que ya conocen de ecología.”

PARTE 3: CONTROL 1 (15 minutos, individual, con calificación)

Aplicar al final de la sesión, después del laboratorio. Los estudiantes pueden usar la tabla ASCII impresa (se les entrega junto con el control). No se permite uso de calculadora ni celular.

Control 1 — Semana 2

Introducción al Análisis de Datos y Programación Nombre: _____

Fecha: _____ Puntaje: _____ / 24

Puedes usar la tabla ASCII que se te entregó. No se permite calculadora ni celular. Tiempo: 15 minutos.

1. (4 puntos) Convierte los siguientes números decimales a binario (8 bits):

a) 42 = _____

b) 100 = _____

2. (4 puntos) Convierte los siguientes números binarios a decimal:

a) 01010101 = _____

b) 11001010 = _____

3. (4 puntos) Usando la tabla ASCII:

a) ¿Cuál es el código decimal y binario de la letra “C”?

Decimal: _____ Binario: _____

b) ¿Qué carácter corresponde al número decimal 115?

Carácter: _____

4. (4 puntos) Un compañero te entrega la siguiente secuencia binaria y te dice que representa dos letras en ASCII:

01001101 01000001

¿Qué letras son? _____

¿Qué palabra podrían formar como iniciales? (respuesta libre) _____

5. (4 puntos) Un píxel de una imagen digital tiene los valores RGB = (0, 128, 255).

a) ¿De qué color aproximado será este píxel? (marca una opción)

☐ Rojo

☐ Verde

☐ Azul cielo

☐ Negro

b) ¿Cuántos bits se necesitan para almacenar UN solo píxel en formato RGB? _____

6. (4 puntos) Responde brevemente:

a) ¿Por qué los computadores usan el sistema binario en lugar del decimal? (2 oraciones máximo)

[espacio]

b) ¿Qué ocurre cuando un programa intenta mostrar texto codificado en UTF-8 pero lo interpreta como ASCII?

[espacio]

Pauta de corrección

Pregunta	Respuesta	Puntos
1a	42 = 00101010	2 pts (1 si proceso correcto pero error menor)
1b	100 = 01100100	2 pts
2a	01010101 = 85	2 pts
2b	11001010 = 202	2 pts
3a	C = 67 decimal, 01000011 binario	2 pts (1 por cada parte)
3b	115 = “s” (minúscula)	2 pts
4	M (77) y A (65) → “MA”	3 pts (1.5 por letra) + 1 pt por respuesta libre coherente
5a	Azul cielo (R=0, G=128, B=255)	2 pts
5b	24 bits (3 bytes × 8 bits)	2 pts
6a	Porque es más fácil/robusto implementar dos estados en hardware (voltaje alto/bajo, conduce/no conduce). Minimiza errores por ruido.	2 pts (1 por mencionar hardware, 1 por robustez/simplicidad)
6b	Los caracteres especiales (ñ, acentos, emojis) se muestran como símbolos incorrectos o signos de interrogación, porque ASCII no incluye esos códigos.	2 pts (1 por mencionar que se muestran mal, 1 por explicar que es por incompatibilidad de tablas)

Notas pedagógicas

Errores esperados en el laboratorio

- **Olvidar los ceros a la izquierda:** escribir 1001000 (7 bits) en vez de 01001000 (8 bits). Resulta en decodificación incorrecta. → Insistir: siempre 8 bits por carácter.
- **Leer la pulsera al revés:** si la pulsera no tiene marca de inicio, el receptor puede leerla de derecha a izquierda. → Convenir marcar el inicio con un nudo o cuenta especial.
- **Confundir mayúsculas y minúsculas:** “a”=97 vs “A”=65. → Oportunidad para mostrar que la diferencia es un solo bit.
- **Buscar la ñ en la tabla ASCII:** no está. → Oportunidad perfecta para introducir UTF-8.

Adaptación para bancas fijas (auditorio)

- Usar **papel cuadriculado** en vez de cuentas (más práctico, menos caos).
- Formato: hoja cuadriculada donde cada fila = 1 byte (8 cuadros). Colorear cuadro = 1, dejar vacío = 0.
- Para el intercambio (Fase 2): pasar la hoja al compañero de la fila de atrás o adelante.
- Para la Fase 3 (mensajes grupales): cada grupo escribe en una sola hoja (5 filas = 5 letras = 5 bytes) y la pasa al grupo receptor.

Conexión con el resto del curso

Concepto de esta semana	Dónde se profundiza
Bit como unidad mínima	Semana 3 (Entropía de Shannon, bits como medida de información)
Conversión entre bases	Semana 8 (Python: bin(), hex(), int())
ASCII / codificación	Semana 8 (strings en Python, encoding)
Píxeles y RGB	Semana 13 (Pandas + matplotlib: visualización de datos)
Muestreo de sonido	Semana 7 (AI y monitoreo acústico en conservación)
Estándares como convención humana	Semana 6 (LLMs: tokenización como otra forma de codificación)